Comparación experimental de los algoritmos de ordenación por inserción y Quicksort según el tamaño y tipo de datos

Experimental comparison of insertion sort and Quicksort by data size and input type

Kevin Xavier Marca Capacuti· Escuela Profesional de Ingeniería en Informática y Sistemas, Facultad de Ingeniería · Universidad Nacional Jorge Basadre Grohmann · Tacna, Perú · kmarcac@unjbg.edu.pe

Resumen. El objetivo de este trabajo fue comparar el tiempo y algunas operaciones de los algoritmos de ordenación por inserción y Quicksort usando distintos tamaños y tipos de datos. Ambos se programaron en C++ con Dev-C++ 5.11 y se probaron con cinco tipos de arreglos: aleatorios, ascendentes, descendentes, casi ordenados y con valores repetidos. Se usaron 1 000, 5 000 y 10 000 elementos, con 30 repeticiones por cada caso (900 ejecuciones en total). Se midieron el tiempo, las comparaciones, los intercambios y la profundidad de la recursión. Con 10 000 datos aleatorios, Quicksort tardó 2,73 ms y la inserción 113,50 ms; es decir, la inserción tardó unas 42 veces más. Con datos descendentes, la diferencia fue de unas 340 veces. En cambio, con datos ya ordenados de menor a mayor, la inserción fue más rápida. En general, Quicksort funcionó mejor en casi todos los casos, mientras que la inserción destacó cuando los datos ya estaban ordenados. Los tiempos pueden variar según la computadora, por lo que es importante repetir las pruebas en condiciones similares

Abstract. This study compared the execution time and operations of insertion sort and Quicksort using different data sizes and input types. Both algorithms were programmed in C++ with Dev-C++ 5.11 and tested with five patterns: random, ascending, descending, nearly sorted, and repeated values. The tests used 1,000, 5,000, and 10,000 elements, with 30 repetitions per case (900 runs in total). Time, comparisons, swaps, and recursion depth were recorded. With 10,000 random values, Quicksort took 2.73 ms, while insertion sort took 113.50 ms. Quicksort was faster in almost every case, except when the data were already sorted in ascending order. Since results can vary depending on the computer, tests should be repeated under similar conditions.

Palabras clave: ordenación por inserción, Quicksort, complejidad algorítmica, evaluación experimental, patrones de entrada

Keywords: insertion sort, Quicksort, algorithmic complexity, experimental evaluation, input patterns.

1. Introducción

Ordenar datos es una tarea común en programación. Por ejemplo, podemos ordenar nombres, precios, notas o resultados de búsqueda. Estudiar estos algoritmos ayuda a entender la complejidad algorítmica, que explica cómo aumenta el trabajo de un programa cuando recibe más datos (Cormen et al., 2022). Aunque la teoría permite estimar ese crecimiento, el tiempo real también depende del tipo de datos y de la computadora; por eso es útil realizar pruebas.

Otros estudios también han comparado algoritmos de ordenación y su eficiencia usando mediciones y análisis de datos (Salgado et al., 2022). En este trabajo se comparan dos métodos. La ordenación por inserción es sencilla: toma un elemento y lo coloca en su posición correcta. Quicksort, propuesto por Hoare (1962), divide los datos alrededor de un valor llamado pivote y ordena cada parte por separado

El objetivo fue comparar el tiempo de ejecución, el número de comparaciones, los cambios de posición y la profundidad de la recursión de ambos algoritmos. Para ello, se usaron cinco tipos de datos y tres tamaños diferentes. Se esperaba que Quicksort fuera más rápido con datos aleatorios y en orden descendente, mientras que la inserción podría funcionar mejor con datos que ya estuvieran ordenados

2. Método

2.1 Algoritmos utilizados. Se probaron versiones sencillas de los dos algoritmos, sin mejoras especiales. En la ordenación por inserción, el programa toma cada elemento desde el segundo y lo compara con los anteriores. Si encuentra valores mayores, los desplaza hacia la derecha hasta dejar el elemento en su lugar. Es parecido a ordenar cartas en la mano. Los valores iguales no se adelantan entre sí. Quicksort elige un elemento como pivote y usa dos índices: uno avanza desde el inicio y otro desde el final. Cuando encuentran valores que deben cambiar de lado, se intercambian. Al terminar esta división, el algoritmo repite el proceso en cada parte mediante llamadas recursivas. La recursión significa que una función vuelve a llamarse para resolver una parte más pequeña del problema. En esta **versión no se añadieron mejoras para limitar la profundidad de esas llamadas**

2.2 Complejidad teórica. La Tabla 1 resume cuánto trabajo pueden necesitar los algoritmos según la teoría (Cormen et al., 2022; Knuth, 1998). En inserción, el mejor caso ocurre cuando los datos ya están ordenados; el peor, cuando están en orden inverso. En Quicksort, el resultado depende de qué tan bien el pivote divide los datos. La memoria extra de inserción es constante porque usa unas pocas variables. Quicksort también necesita espacio para guardar las llamadas **recursivas, por lo que ese espacio depende de cuántos niveles de llamadas se formen.**

Tabla 1. Complejidad teórica de los algoritmos. Fuente: elaboración propia a partir de Cormen et al. (2022) y Knuth (1998).

algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria extra	Estabilidad
Insercion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Estable
quicksort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) en promedio	Inestable

2.3 Entorno de pruebas. Las pruebas se realizaron en Dev-C++ 5.11, que tiene como versión del compilador 4.9.2. Todas las pruebas deben hacerse en el mismo equipo y conviene cerrar otros programas para reducir posibles cambios en los tiempos

2.4 Creación de los datos. Los arreglos se formaron con números enteros usando srand() y rand(). Se eligió una semilla diferente para cada repetición mediante la fórmula **42 + 7n + 100p** + r. La semilla sirve para generar una secuencia de números y poder repetir la prueba en las mismas condiciones. Ambos programas usaron la misma fórmula, por lo que cada algoritmo recibió los mismos datos. Se probaron cinco patrones: (1) aleatorio, con números generados al azar; (2) ascendente, del menor al mayor; (3) descendente, del mayor al menor; (4) casi

ordenado, que empieza ordenado y luego se cambian al azar algunas posiciones equivalentes al 5 % de n; y (5) con duplicados, donde solo se usan 15 valores posibles. En este entorno, rand() genera valores hasta 32 767, por lo que el conjunto aleatorio puede contener números repetidos

2.5 Tamaños y repeticiones. Se probaron arreglos de 1 000, 5 000 y 10 000 elementos. Cada combinación de algoritmo, tipo de datos y tamaño se ejecutó 30 veces. Como se compararon dos algoritmos, cinco patrones, tres tamaños y 30 repeticiones, se realizaron 900 ejecuciones en total. Después de cada prueba se revisó que el arreglo estuviera ordenado correctamente

2.6 Qué se midió y cómo. El tiempo se midió en milisegundos con **std::chrono::steady_clock,** tomando solo el tiempo que tardaba el algoritmo en ordenar, sin contar la escritura de resultados. También se contaron las comparaciones entre elementos y los cambios de posición. En inserción, la columna llamada “intercambios” representa desplazamientos de elementos; en Quicksort representa cambios entre dos posiciones distintas, así que no son operaciones exactamente iguales. La profundidad de recursión indica cuántas llamadas de Quicksort pueden estar activas una dentro de otra. La memoria extra no se midió directamente: se estimó según la teoría y la profundidad observada. Los resultados se guardaron en archivos CSV, que son archivos de texto organizados en columnas y pueden abrirse con Excel. Luego se calcularon la media (promedio), la mediana (valor central), la desviación estándar (qué tanto varían los tiempos), el mínimo y el máximo

3. Resultados

La Tabla 2 presenta los tiempos obtenidos en las pruebas. La Figura 1 permite comparar cómo cambia el tiempo al aumentar la cantidad de datos. La Figura 2 muestra la variación de los tiempos en las 30 repeticiones para arreglos de 10 000 elementos. La Tabla 4 reúne las comparaciones, los cambios de posición y la profundidad de recursión; la Tabla 5 indica cuántas veces tardó inserción en comparación con Quicksort

Tabla 2. Estadística descriptiva del tiempo de ejecución (ms) por tamaño, algoritmo y patrón (30 repeticiones por fila). METODO INSERCION

n	patron	Media	Mediana	Desv. est.	Máx.	Mín.
1000	aleatorio	1.79	1.80	0.21	2.21	1.24
1000	ascendente	0.01	0.01	0.00	0.01	0.01
1000	casi_ordenado	0.17	0.15	0.04	0.35	0.14
1000	descendente	2.42	2.32	0.30	3.18	2.06
1000	duplicados	1.10	1.02	0.23	1.83	0.95
5000	aleatorio	33.11	34.17	3.72	39.07	26.41
5000	ascendente	0.02	0.02	0.00	0.03	0.02
5000	casi_ordenado	3.62	3.63	0.22	4.02	3.21
5000	descendente	56.84	55.80	4.07	70.61	52.85
5000	duplicados	26.45	26.03	1.25	29.46	25.11
10000	aleatorio	113.50	112.37	7.53	137.48	105.66
10000	ascendente	0.05	0.04	0.04	0.27	0.04
10000	casi_ordenado	16.67	16.35	2.44	21.19	13.13
10000	descendente	222.87	219.32	9.74	245.24	212.47
10000	duplicados	103.67	102.47	5.18	126.51	98.85
Total		**38.82**	**3.63**	**61.30**	**245.24**	**0.01**

Tabla 3. Estadística descriptiva del tiempo de ejecución (ms) por tamaño, algoritmo y patrón (30 repeticiones por fila). METODO QUICKSORT

n	patron	Media	Mediana	Desv. est.	Máx.	Mín.
1000	aleatorio	0.21	0.20	0.03	0.32	0.13
1000	ascendente	0.04	0.04	0.01	0.06	0.04
1000	casi_ordenado	0.05	0.05	0.01	0.07	0.04
1000	descendente	0.04	0.04	0.01	0.09	0.04
1000	duplicados	0.11	0.09	0.07	0.48	0.09
5000	aleatorio	0.76	0.74	0.06	0.96	0.69
5000	ascendente	0.23	0.21	0.05	0.40	0.19
5000	casi_ordenado	0.41	0.46	0.11	0.64	0.25
5000	descendente	0.25	0.23	0.05	0.44	0.22
5000	duplicados	1.01	0.99	0.10	1.28	0.83
10000	aleatorio	2.73	2.70	0.21	3.15	2.41
10000	ascendente	0.81	0.79	0.10	1.06	0.67
10000	casi_ordenado	0.59	0.58	0.04	0.75	0.53
10000	descendente	0.66	0.65	0.16	1.07	0.45
10000	duplicados	1.09	1.06	0.09	1.33	1.01
Total		**0.60**	**0.46**	**0.67**	**3.15**	**0.04**

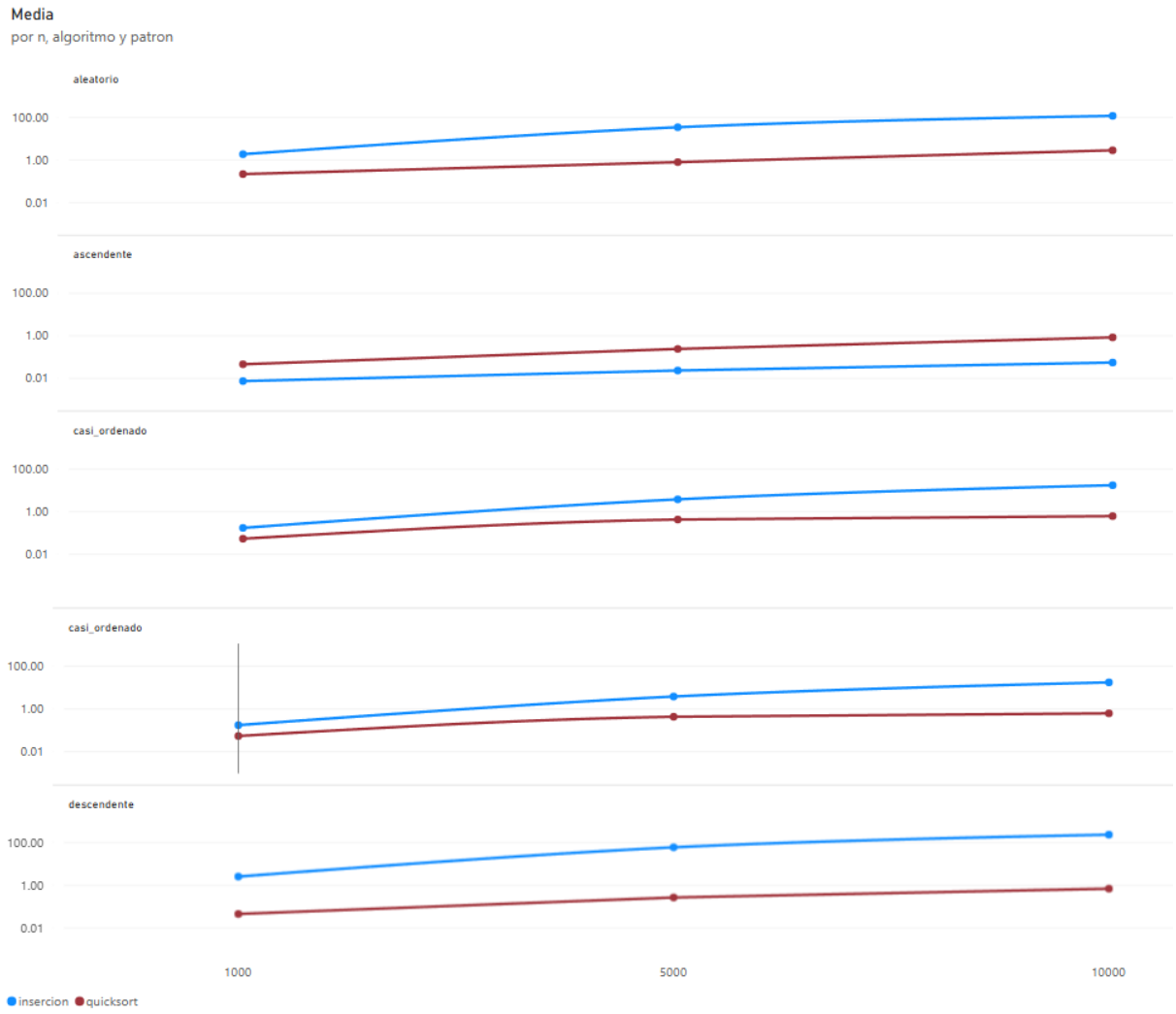


Figura 1. Tiempo medio de ejecución según n para cada patrón (escalas logarítmicas)

3.1 Tiempos de ejecución. Con 10 000 datos aleatorios, Quicksort tardó 2,73 ms y la inserción 113,50 ms. Esto significa que inserción tardó unas 42 veces más. Con datos en orden descendente, inserción tardó 222,87 ms y Quicksort 0,656 ms, una diferencia de unas 340 veces. Con datos ascendentes ocurrió lo contrario: inserción tardó 0,053 ms y Quicksort 0,808 ms, por lo que inserción fue aproximadamente 15 veces más rápida. Con datos casi ordenados, inserción tardó 16,67 ms frente a 0,590 ms de Quicksort. Con valores repetidos, tardó 103,67 ms frente a 1,09 ms de Quicksort

Cuando el tamaño aumentó de 1 000 a 10 000 elementos, es decir, diez veces más datos, el tiempo de inserción aumentó mucho más que el de Quicksort en los arreglos desordenados. Esto coincide con la teoría: en general, inserción puede necesitar un trabajo cercano a n^2, mientras que Quicksort suele crecer alrededor de $n \log_2 n$. Dicho de forma sencilla, al aumentar los datos, el trabajo de inserción crece más rápido en muchos casos

Tabla 4. Comparaciones, intercambios (en la inserción, desplazamientos) y profundidad de recursión (promedios), n = 10 000. Fuente: elaboración propia.

Algoritmo	Patrón	Comparaciones	Intercambios	Prof. recursión
Inserción	Aleatorio	25 036636	25 026 646	--

Inserción	Ascendente	9999	0	--
Inserción	Descendente	49 995 000	49 995 000	--
Inserción	Casi ordenado	3 137 569	3127 570	--
Inserción	Duplicados	23 341 406	23 331 409	--
Quicksort	Aleatorio	182 090	31 457	30,4
Quicksort	Ascendente	143 615	0	14,0
Quicksort	Descendente	143614	5000	14,0
Quicksort	Casi ordenado	145 674	2 296	16,5
Quicksort	Duplicados	148 070	54 355	20,6

3.2 Comparaciones, movimientos y recursión. Los conteos coincidieron con lo que se espera de la teoría. Cuando los datos estaban ordenados de menor a mayor, inserción realizó $n - 1$ comparaciones y no necesitó mover elementos. En cambio, con los datos en orden inverso, tuvo que comparar y desplazar muchos elementos: para 10 000 datos hizo 49 995 000 comparaciones. Con datos aleatorios también realizó millones de comparaciones. Los arreglos casi ordenados necesitaron menos trabajo que los aleatorios, pero todavía mucho más que los que ya estaban ordenados

En Quicksort con datos aleatorios, el número de comparaciones aumentó aproximadamente de acuerdo con $n \log_2 n$. La profundidad media de recursión también aumentó con el tamaño del arreglo, y el máximo observado fue de 34 niveles. Con datos ascendentes y descendentes, el pivote central dividió los arreglos de forma bastante equilibrada. En el caso ascendente no hubo intercambios; en el descendente hubo 5 000. La inserción usa unas pocas variables adicionales, mientras que Quicksort necesita guardar las llamadas que todavía debe resolver

Tabla 5. Veces que la inserción tarda más que Quicksort (tiempo medio de la inserción / tiempo medio de Quicksort). Un valor menor que 1 significa que la inserción fue más rápida

Patrón	n = 1000	n = 5000	n = 10000
Aleatorio	8,6	43,6	41,6
Ascendente	0,16	0,10	0,07
Descendente	56,5	2,24	340
Casi ordenado	3,2	8,9	28,3
Duplicados	10,3	26,2	95,5

3.3 Diferencias entre repeticiones. En la mayoría de las pruebas, los tiempos de las 30 repeticiones fueron similares. Sin embargo, algunas ejecuciones tardaron más que otras. Por ejemplo, con 5 000 datos casi ordenados, Quicksort tuvo tiempos distintos entre la primera parte y la última parte de las repeticiones. Como el número de comparaciones y cambios fue el mismo, es posible que la diferencia se debiera a otros procesos de la computadora o al sistema

operativo. Por esa razón también se informa la mediana, que se ve menos afectada por valores muy altos o muy bajos.

4. Discusión

Los resultados fueron parecidos a lo que indica la teoría. Inserción necesitó pocas comparaciones cuando los datos ya estaban ordenados, pero muchas cuando estaban en orden inverso o mezclados. Quicksort, en general, necesitó menos comparaciones y su tiempo aumentó más lentamente al crecer el arreglo. La hipótesis de que Quicksort sería más rápido con datos aleatorios y descendentes se cumplió. La idea de que inserción también sería mejor con datos casi ordenados solo se cumplió en parte: en estas pruebas fue más lenta que Quicksort. Aunque se cambiaron de lugar solo el 5 % de los elementos, algunos estaban lejos de su posición correcta y tuvieron que desplazarse varios valores.

Estabilidad y valores repetidos. Un algoritmo estable conserva el orden original de elementos que tienen el mismo valor o clave. Inserción es estable, pero esta versión de Quicksort no garantiza esa propiedad. Esto puede ser importante al ordenar datos con más de una característica, por ejemplo, primero por nota y luego por apellido. En este trabajo se ordenaron números, así que la estabilidad no cambió el resultado numérico. Cuando se usaron solo 15 valores distintos, Quicksort siguió siendo rápido. Sus índices se detienen al encontrar valores iguales al pivote, lo que ayuda a repartir el arreglo, aunque realizó más intercambios que con datos aleatorios. Inserción siguió tardando más porque muchos elementos debían desplazarse.

Limitaciones del trabajo. Los tiempos dependen de la computadora, el compilador y el sistema operativo, así que pueden cambiar en otro equipo. Solo se probaron tres tamaños y números enteros. Además, rand() tiene un límite en este entorno, por lo que algunos valores del patrón aleatorio se repitieron. Quicksort tampoco tiene un límite fijo de profundidad en esta versión; aunque el máximo observado fue 34 niveles, no se probaron arreglos preparados para causar muchas llamadas recursivas. Los movimientos de inserción y los intercambios de Quicksort no son exactamente la misma operación. Por último, otros programas abiertos pudieron afectar los tiempos, sobre todo cuando eran menores de un milisegundo. Por eso conviene prestar más atención a las diferencias grandes que a las pequeñas.

5. Conclusiones

- Quicksort fue unas 42 veces más rápido que inserción con 10 000 datos aleatorios: tardó 2,73 ms frente a 113,50 ms. Con datos descendentes, fue unas 340 veces más rápido.
- Inserción fue más rápida cuando los datos ya estaban ordenados de menor a mayor: tardó 0,053 ms frente a 0,808 ms de Quicksort. Con datos casi ordenados, en cambio, fue unas 28 veces más lenta.
- Los resultados de las comparaciones coincidieron con lo esperado: inserción necesita poco trabajo si los datos están ordenados, pero mucho más si están desordenados. Quicksort tuvo, en general, un crecimiento menor del trabajo y una profundidad de recursión reducida en las pruebas.

- Quicksort siguió funcionando bien con datos que tenían solo 15 valores diferentes. Inserción tiene la ventaja de ser estable, es decir, conserva el orden relativo de los elementos iguales; Quicksort no garantiza eso en esta versión.
- En general, Quicksort conviene para arreglos medianos o grandes que están desordenados. Inserción puede ser una buena opción para arreglos pequeños o que ya están ordenados. Para confirmar los resultados, sería útil repetir las pruebas en condiciones controladas y probar tamaños mayores.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal, 5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Knuth, D. E. (1998). *The art of computer programming, Vol. 3: Sorting and searching* (2.ª ed.). Addison-Wesley.

Murillo, J., & Caamaño, S. (2013). Comparación entre algoritmos recursivos e iterativos y su medición en términos de eficiencia. *Uniciencia, 27*(1), 341–350. https://www.redalyc.org/pdf/4759/475947762020.pdf

Salgado, M., Pérez, J., & Albarrán, S. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia E Ingeniería, 43*(03), 273–280. https://doi.org/10.53766/cei/2022.43.03.005